

YZ 203 - Week 3: Lab / Practice Session Notes

Lab Objectives

This lab goes from the simplest possible Timer to a first playable screen of the egg game. Work through the exercises in order; each one adds exactly one idea to the previous one.

By the end of this session, the student will be able to:

- Create, start, stop and reconfigure a `javax.swing.Timer`
- Explain why a Timer is used instead of a loop with `Thread.sleep()`
- Move a component with `null` layout and `setLocation()` / `setBounds()`
- Reverse direction at a wall and restart an object at a random position
- Combine Timer with JButton, JSlider, JProgressBar and JLabel
- Move state and behavior into a small class (`Egg`)

Level	Exercises
Easy (warm-up)	1. Ticker, 2. Digital clock, 3. Start / Stop, 4. Auto progress bar
Medium	5. Move right, 6. Bounce, 7. Random fall, 8. Speed slider
Hard	9. Egg class, 10. Falling Egg v0, 11. Challenge: two eggs

Duration: about 2 class hours. **Required materials:** JDK and NetBeans from Week 1.

Every exercise assumes the file starts with `import javax.swing.*;` — with only that import, `Timer` means `javax.swing.Timer`.

Exercise 1 (Easy): A Label That Counts by Itself

Goal: Create your first `Timer`. No button, no click: the clock is the event source.

```
import javax.swing.*;

public class TickDemo {
    static int seconds = 0;
```

```

public static void main(String[] args) {
    JFrame f = new JFrame("Ticker");
    JLabel label = new JLabel("Seconds: 0", SwingConstants.CENTER);

    Timer timer = new Timer(1000, e -> {
        seconds++;
        label.setText("Seconds: " + seconds);
    });
    timer.start();

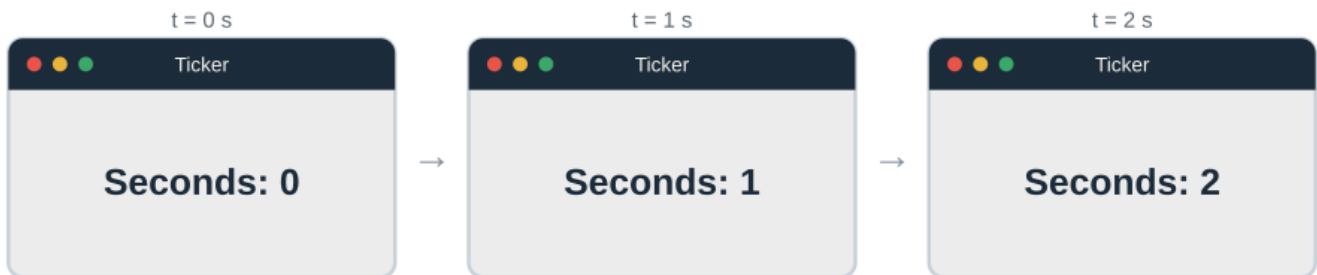
    f.add(label);
    f.setSize(250, 120);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
}
}

```

Step by step:

1. `new Timer(1000, ...)` creates a timer that fires every 1000 ms. It does nothing yet.
2. The lambda is the `ActionListener`: it runs once per tick.
3. `timer.start()` begins ticking. The counter lives in a `static` field because a lambda cannot change an ordinary local variable.

Expected output:



Try it: change `1000` to `250` and watch the counter run four times faster.

Exercise 2 (Easy): A Digital Clock

Goal: Same Timer, but the label now shows the real time. The state is no longer our own counter: we read it from the system.

```

import javax.swing.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ClockDemo {
    public static void main(String[] args) {
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("HH:mm:ss");
        JFrame f = new JFrame("Clock");
        JLabel label = new JLabel(LocalTime.now().format(fmt),
        SwingConstants.CENTER);
        label.setFont(label.getFont().deriveFont(36f));

        new Timer(1000, e ->
label.setText(LocalTime.now().format(fmt))).start();

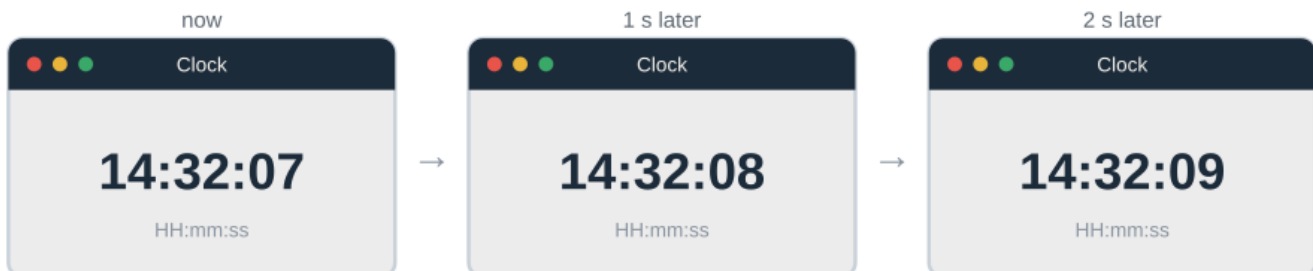
        f.add(label);
        f.setSize(260, 120);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}

```

Step by step:

1. `LocalTime.now()` returns the current time; `format(fmt)` turns it into text such as `14:32:07`.
2. We set the first text ourselves so the clock is not empty during the first second.
3. `deriveFont(36f)` makes a bigger copy of the label's font.

Expected output:



Try it: use the pattern `"HH:mm"` and notice that the label still updates every second, even though the text changes only once a minute.

Exercise 3 (Easy): Start and Stop Buttons

Goal: Keep a reference to the Timer so that two buttons can control it.

```
import javax.swing.*;

public class StartStopDemo {
    static int count = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Start / Stop");
        JPanel p = new JPanel();
        JLabel label = new JLabel("Count: 0");
        JButton start = new JButton("Start");
        JButton stop = new JButton("Stop");

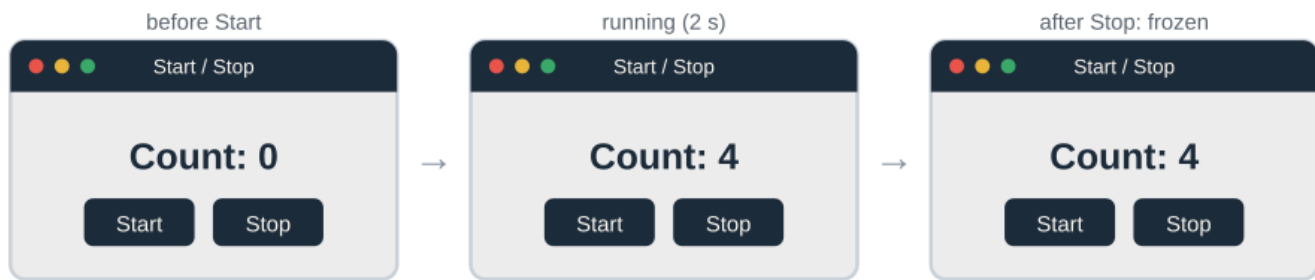
        Timer timer = new Timer(500,
            e -> label.setText("Count: " + (++count)));
        start.addActionListener(e -> timer.start());
        stop.addActionListener(e -> timer.stop());

        p.add(label); p.add(start); p.add(stop);
        f.add(p);
        f.setSize(300, 120);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. The Timer is created but not started, so the label stays at `Count: 0`.
2. The Start button calls `timer.start()`, the Stop button calls `timer.stop()`.
3. After Stop the count keeps its value. Pressing Start again continues from there.

Expected output:



Try it: press Start twice in a row. Does the counter speed up? Why or why not? (Look at what `start()` does to a timer that is already running.)

Exercise 4 (Easy): A Progress Bar That Fills Itself

Goal: Let a timer stop itself from inside its own listener.

```
import javax.swing.*;

public class AutoProgress {
    static int value = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Loading");
        JProgressBar bar = new JProgressBar(0, 100);
        bar.setStringPainted(true);

        Timer timer = new Timer(50, null);
        timer.addActionListener(e -> {
            value++;
            bar.setValue(value);
            if (value >= 100) timer.stop();
        });
        timer.start();

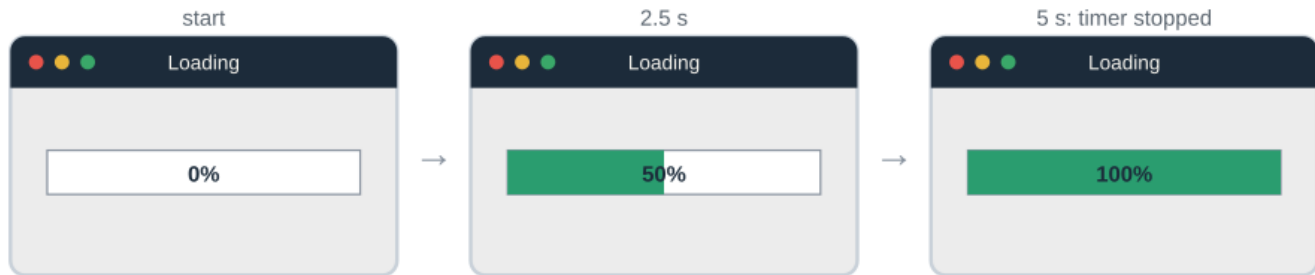
        f.add(bar);
        f.setSize(300, 90);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. 100 ticks of 50 ms fill the bar in 5 seconds.

2. `new Timer(50, null)` creates the timer without a listener. We add one on the next line because a lambda cannot refer to `timer` inside its own declaration.
3. When `value` reaches 100 the listener calls `timer.stop()`, so the timer ends itself.

Expected output:



Try it: remove the `if (value >= 100) timer.stop();` line. What does `setValue(101)` do to the bar, and why is it still a bad idea to leave the timer running?

Exercise 5 (Easy to Medium): A Box Slides Across the Window

Goal: First real animation. Combine `null` layout (Week 1) with a Timer: change a number, move the component.

```
import javax.swing.*;
import java.awt.Color;

public class MoveRight {
    static int x = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Move Right");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
        box.setBounds(x, 50, 40, 40);
        panel.add(box);

        new Timer(16, e -> {
            x += 3;
            box.setLocation(x, 50);
        }).start();

        f.add(panel);
    }
}
```

```

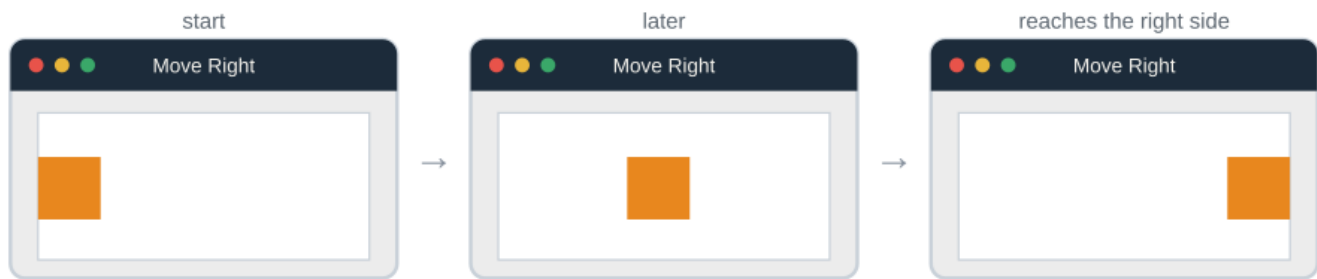
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}

```

Step by step:

1. `new JPanel(null)` turns the layout manager off, so `setBounds()` and `setLocation()` are respected.
2. An opaque `JLabel` with a background color is the simplest colored rectangle in Swing.
3. Every 16 ms (about 60 times a second) `x` grows by 3 and the box is moved to the new position.

Expected output:



Try it: the box eventually leaves the window and keeps going. Do not fix it yet; that is exactly what Exercise 6 solves.

Exercise 6 (Medium): Bouncing Off the Walls

Goal: Add the first piece of game logic: a rule that depends on the current position.

```

import javax.swing.*;
import java.awt.Color;

public class BounceDemo {
    static int x = 0, dx = 4;

    public static void main(String[] args) {
        JFrame f = new JFrame("Bounce");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
    }
}

```

```

    box.setBounds(0, 50, 40, 40);
    panel.add(box);

    new Timer(16, e -> {
        x += dx;
        if (x < 0 || x > panel.getWidth() - 40) dx = -dx;
        box.setLocation(x, 50);
    }).start();

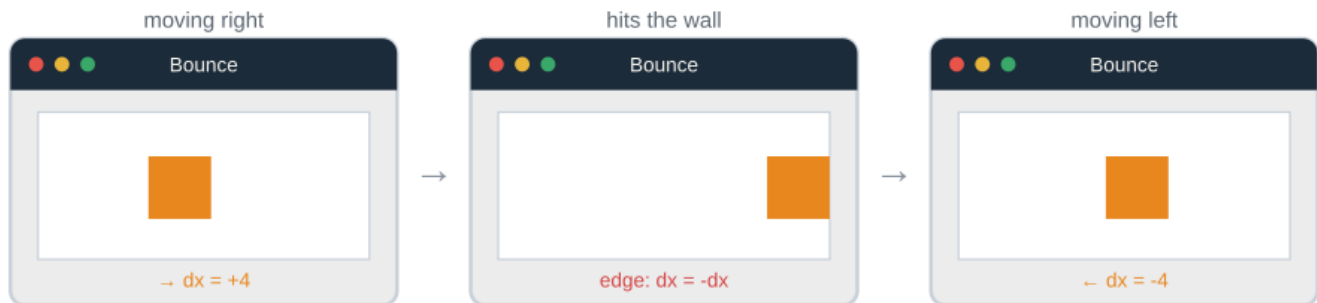
    f.add(panel);
    f.setSize(420, 200);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
}
}

```

Step by step:

1. `dx` is the step per tick: `+4` moves right, `-4` moves left.
2. When the box passes either wall, `dx = -dx` flips the direction.
3. `panel.getWidth() - 40` is the right wall minus the box width. It adapts when you resize the window.

Expected output:



Try it: replace the condition with `x == 0 || x == panel.getWidth() - 40`, then resize the window to different widths. For some widths the box escapes. Why? (Hint: with a step of 4, can `x` skip the exact value?)

Exercise 7 (Medium): A Falling Egg with a Random Column and a Miss Counter

Goal: Use `Random` and a score label. This is the heart of the game: an egg falls, and if it reaches the bottom it counts as missed.

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class RandomFall {
    static int y = 0, missed = 0;

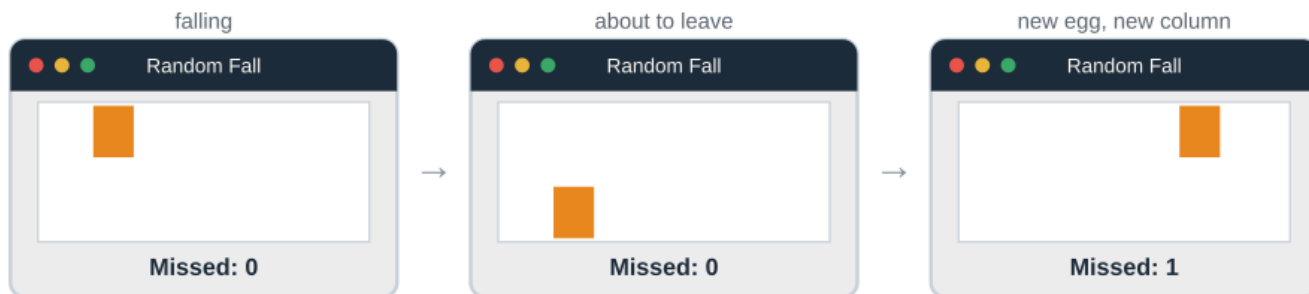
    public static void main(String[] args) {
        Random rnd = new Random();
        JFrame f = new JFrame("Random Fall");
        JPanel field = new JPanel(null);
        JLabel egg = new JLabel();
        egg.setOpaque(true);
        egg.setBackground(Color.ORANGE);
        field.add(egg);
        JLabel missedLabel = new JLabel("Missed: 0", SwingConstants.CENTER);

        new Timer(16, e -> {
            y += 4;
            if (y > field.getHeight()) {
                missed++;
                missedLabel.setText("Missed: " + missed);
                y = 0;
                egg.setLocation(rnd.nextInt(field.getWidth() - 30), 0);
            }
            egg.setBounds(egg.getX(), y, 30, 40);
        }).start();

        f.add(field);
        f.add(missedLabel, BorderLayout.SOUTH);
        f.setSize(260, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `y` grows by 4 each tick, so the egg falls. Swing's `y` axis points down: 0 is the top.
2. When `y` passes the field height, the egg counts as missed, goes back to the top and gets a new random `x`.
3. `rnd.nextInt(field.getWidth() - 30)` returns a number from 0 up to, but not including, the width minus the egg width, so the egg always stays inside.

Expected output:

Try it: make the egg faster after every miss by changing `y += 4` to `y += 4 + missed`. What happens after about ten misses?

Exercise 8 (Medium): A Speed Slider

Goal: Connect Week 2 and Week 3: a `JSlider` (`ChangeListener`) changes the delay of a running `Timer` (`ActionListener`).

```
import javax.swing.*;
import java.awt.*;

public class SpeedDemo {
    static int x = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Speed");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
        box.setBounds(0, 40, 40, 40);
        panel.add(box);
        JSlider speed = new JSlider(5, 100, 30);
```

```

    Timer timer = new Timer(30, e -> {
        x = (x + 4) % panel.getWidth();
        box.setLocation(x, 40);
    });
    speed.addChangeListener(e -> timer.setDelay(speed.getValue()));
    timer.start();

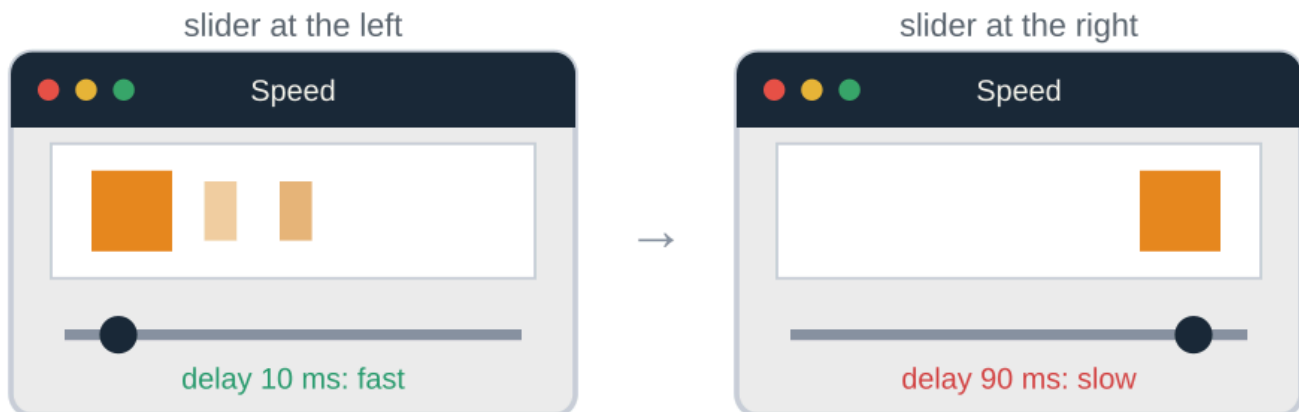
    f.add(panel);
    f.add(speed, BorderLayout.SOUTH);
    f.setSize(420, 200);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
}
}

```

Step by step:

1. The slider range 5 to 100 is the delay in milliseconds.
2. `timer.setDelay(...)` takes effect immediately, with no restart.
3. `% panel.getWidth()` wraps the box back to the left edge instead of letting it leave.

Expected output:



Try it: the slider feels backwards, because a larger value means a slower box. Change the code so that dragging right makes the box faster. (Hint: the delay is `105 - speed.getValue()`.)

Exercise 9 (Medium to Hard): Your First Own Class, Egg

Goal: Stop scattering `x`, `y` and `speed` as static variables. Give them to an `Egg` object that also knows how to fall. The screen looks exactly the same as before; only the structure of the code improves.

```
import javax.swing.*;
import java.awt.Color;

class Egg {
    private int y = 0;
    private final int x, speed;
    private final JLabel view = new JLabel();

    Egg(int x, int speed) {
        this.x = x;
        this.speed = speed;
        view.setOpaque(true);
        view.setBackground(Color.ORANGE);
        view.setBounds(x, 0, 30, 40);
    }

    void fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) y = 0;
        view.setLocation(x, y);
    }

    JLabel getView() { return view; }
}

public class EggClassDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Egg class");
        JPanel field = new JPanel(null);
        Egg egg = new Egg(110, 4);
        field.add(egg.getView());

        new Timer(16, e -> egg.fall(field.getHeight())).start();

        f.add(field);
        f.setSize(260, 340);
    }
}
```

```

        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}

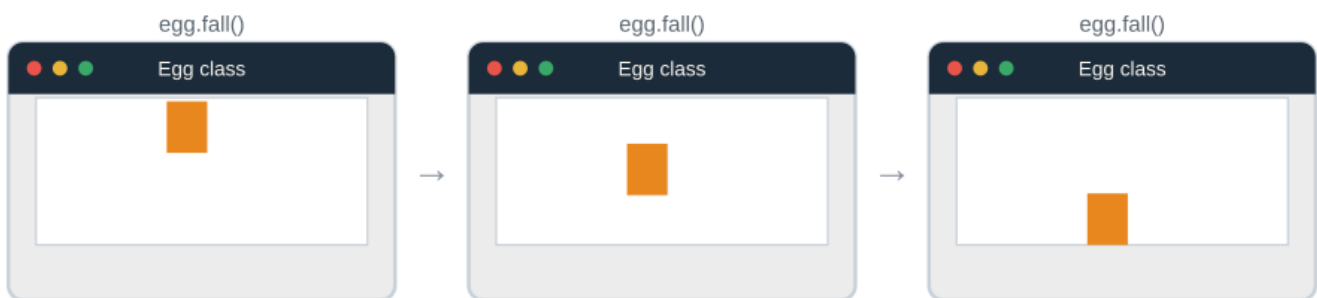
```

Save both classes in one file named `EggClassDemo.java`. Only the class that contains `main` is `public`.

Step by step:

1. The fields are `private`: nothing outside `Egg` can change `y` directly. This is encapsulation.
2. `fall()` is the only door that changes the state. The Timer just asks the egg to fall; the egg knows how.
3. `getView()` hands out the `JLabel` so the panel can display it. The egg is not a component itself; it **has** one (composition).

Expected output:



Try it: create a second egg, `new Egg(40, 7)`, add its view to the panel and call its `fall()` in the same Timer. Notice how little new code a second egg needs.

Exercise 10 (Hard): Falling Egg v0

Goal: Put everything together: a falling egg, a Pause / Start button, a speed slider and a missed counter. This is the first version of the game screen. The egg is not catchable yet; the basket comes in Week 5.

```

import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class FallingEgg {
    static int y = 0, missed = 0;

```

```

static final Random rnd = new Random();

public static void main(String[] args) {
    JFrame f = new JFrame("Falling Egg v0");
    JPanel field = new JPanel(null);
    JLabel egg = new JLabel();
    egg.setOpaque(true);
    egg.setBackground(Color.ORANGE);
    egg.setBounds(100, 0, 30, 40);
    field.add(egg);

    JPanel bar = new JPanel();
    JButton toggle = new JButton("Pause");
    JSlider speed = new JSlider(5, 40, 16);
    JLabel missedLabel = new JLabel("Missed: 0");
    bar.add(toggle);
    bar.add(new JLabel("Delay:"));
    bar.add(speed);
    bar.add(missedLabel);

    Timer timer = new Timer(16, e -> {
        y += 4;
        if (y > field.getHeight()) {
            missed++;
            missedLabel.setText("Missed: " + missed);
            y = 0;
            egg.setLocation(rnd.nextInt(field.getWidth() - 30), 0);
        }
        egg.setLocation(egg.getX(), y);
    });
    toggle.addActionListener(e -> {
        if (timer.isRunning()) {
            timer.stop();
            toggle.setText("Start");
        } else {
            timer.start();
            toggle.setText("Pause");
        }
    });
    speed.addChangeListener(e -> timer.setDelay(speed.getValue()));

    f.add(field);
    f.add(bar, BorderLayout.SOUTH);
}

```

```

        f.setSize(420, 400);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        timer.start();
    }
}

```

Step by step:

1. Two panels: a `null` -layout field for the egg and a normal `FlowLayout` bar for the controls (Week 1 hierarchy: frame, panels, components).
2. One Timer, three listeners: the tick (`ActionListener`), the button (`ActionListener`) and the slider (`ChangeListener`).
3. `timer.isRunning()` decides what the button does, so we never keep our own "paused" flag.

Expected output:



Try it: while the game is paused, drag the slider and then press Start. Does the new speed apply? Explain why using what you know about `setDelay()`.

Exercise 11 (Challenge): Two Eggs, Two Speeds

Goal: Reuse the `Egg` class for several objects with one Timer. The skeleton below compiles and runs, but the eggs stand still until you finish the two TODOs.

```

import javax.swing.*;
import java.awt.*;

class Egg {
    private int y = 0;
    private final int x, speed;
    private final JLabel view = new JLabel();

    Egg(int x, int speed, Color color) {
        this.x = x;
        this.speed = speed;
        view.setOpaque(true);
        view.setBackground(color);
        view.setBounds(x, 0, 30, 40);
    }

    // returns true when the egg wrapped back to the top
    boolean fall(int fieldHeight) {
        y += speed;
        boolean wrapped = y > fieldHeight;
        if (wrapped) y = 0;
        view.setLocation(x, y);
        return wrapped;
    }

    JLabel getView() { return view; }
}

public class TwoEggs {
    static int missed = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Two Eggs");
        JPanel field = new JPanel(null);
        JLabel missedLabel = new JLabel("Missed: 0", SwingConstants.CENTER);

        Egg orange = new Egg(50, 4, Color.ORANGE);
        Egg blue = new Egg(150, 2, new Color(58, 123, 213));
        field.add(orange.getView());
        field.add(blue.getView());

        new Timer(16, e -> {
            // TODO 1: make both eggs fall (pass field.getHeight())

```

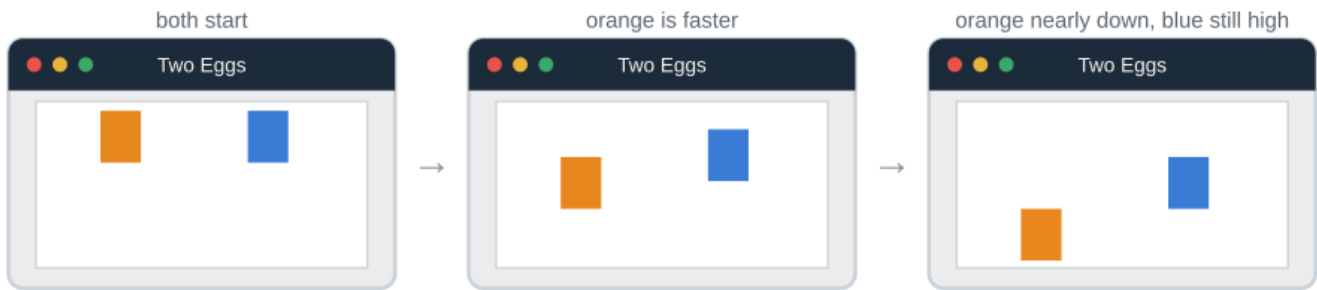
```

        // TODO 2: if an egg wrapped, increase missed and update
missedLabel
        }).start();

        f.add(field);
        f.add(missedLabel, BorderLayout.SOUTH);
        f.setSize(260, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}

```

Expected output once both TODOs are done:



The orange egg falls twice as fast as the blue one, and the `Missed` label counts a miss for each of them.

Hints:

1. `fall()` returns a `boolean`. You can use it directly: `if (orange.fall(h)) missed++;`
2. Do not call `fall()` twice on the same egg in one tick, or it moves twice as far.
3. Think ahead: what would you need if there were 20 eggs? That question is why next weeks introduce `ArrayList`.

Behind the Scenes: OOP in This Lab

Idea	Where you saw it
Interface	Every <code>e -> ...</code> is an <code>ActionListener</code> . A <code>Timer</code> accepts the same listener type as a <code>JButton</code> (Exercises 1, 3), and the slider's <code>ChangeListener</code> is another contract (Exercises 8, 10).

Idea	Where you saw it
Composition	A <code>Timer</code> is not a component. It <i>has</i> listeners and is never <code>add()</code> -ed to a panel. <code>Egg</code> is not a <code>JLabel</code> , it <i>has</i> one (Exercises 9, 11).
Encapsulation	<code>isRunning()</code> , <code>setDelay()</code> and <code>start()</code> hide how the <code>Timer</code> keeps time. <code>Egg</code> hides <code>y</code> behind <code>fall()</code> (Exercises 3, 9).
Inheritance	A plain <code>JLabel</code> becomes a colored box through <code>setOpaque()</code> and <code>setBackground()</code> , both inherited from <code>JComponent</code> (Exercises 5-7).

A Swing `Timer` fires its listener on Swing's event thread, so it is safe to change components from inside it. A `java.util.Timer` runs on its own thread and is not safe for this. That is why we always use `javax.swing.Timer` here, and never `Thread.sleep()` inside a listener.

Homework / Research Questions

No answers are given here. Work them out and bring them to the next session.

1. Build it. Extend Exercise 10 with a `JSpinner` (range 1 to 9) labeled "Lives" and a "Reset" `JBUTTON`. Each missed egg should cost one life instead of only increasing the missed counter. When the lives reach zero, stop the `Timer` and show "Game Over" in a `JLabel`. Reset must restore the lives from the spinner, clear the counter and restart the `Timer`. Which of your listeners needs access to the `timer` variable, and how did you make that possible without a compile error?

2. Investigate. `javax.swing.Timer` has a method `setCoalesce(boolean)`, and it is `true` by default. Read its documentation and answer: what happens if the listener of a 16 ms timer sometimes takes 100 ms to finish, for example because it does heavy work? How would the egg's motion look with coalescing on, and how with it off? Support your answer with a tiny experiment: add `Thread.sleep(100)` once every 50 ticks inside the listener (only for this experiment) and watch the egg.

Submission Checklist

- ☐ `TickDemo.java`: the label counts seconds by itself

- ☐ `ClockDemo.java` : shows the real time and updates every second
- ☐ `StartStopDemo.java` : Start and Stop control the counter
- ☐ `AutoProgress.java` : the bar fills in 5 seconds and the timer stops itself
- ☐ `MoveRight.java` : the box slides right with `null` layout
- ☐ `BounceDemo.java` : the box reverses at both walls, also after resizing
- ☐ `RandomFall.java` : the egg restarts in a random column and misses are counted
- ☐ `SpeedDemo.java` : the slider changes the speed while running
- ☐ `EggClassDemo.java` : state and `fall()` live inside `Egg`
- ☐ `FallingEgg.java` : Pause / Start, speed slider and missed label all work
- ☐ `TwoEggs.java` : both TODOs done, two speeds, shared counter
- ☐ Can explain why a loop with `Thread.sleep()` freezes the window and a `Timer` does not
- ☐ Both homework questions answered

Next Week

Week 4 replaces our colored labels with real drawing: a `JPanel` subclass, `paintComponent()` and `Graphics`. The egg and the basket become shapes, and inheritance with method overriding takes the lead role.